

Raport Techniczny MVP

Bot czy człowiek? Behawioralna biometria ruchu myszy
dla ochrony aplikacji webowej

Piotr Kosiara
Emilia Anczarska

16 maja 2026

Spis treści

1	Wprowadzenie i motywacja	4
1.1	Kontekst problemu	4
1.2	Dlaczego behawioralna biometria myszy	4
1.3	Cel projektu	4
1.4	Zakres i świadome uproszczenia MVP	4
2	Architektura systemu	5
2.1	Widok komponentów	5
2.2	Przepływ danych end-to-end	5
2.3	Diagram przypadków użycia	6
2.4	Model danych (ERD)	7
3	Dokumentacja techniczna codebase	7
3.1	Struktura repozytorium	7
3.2	Frontend testbed (Vite + React + TypeScript)	8
3.2.1	Rola frontendu	8
3.2.2	Wygląd i funkcje interfejsu	8
3.2.3	Zbierane eventy i batching	9
3.2.4	Fingerprint klienta	10
3.3	Pozyskanie danych treningowych (human vs bot)	10
3.3.1	Jak zbierano dane klasy human	10
3.3.2	Jak zbierano dane klas botowych	10
3.3.3	Udział scenariuszy botowych	10
3.3.4	Charakterystyka botów i zamysł projektowy	10
3.4	Backend Collector API (FastAPI)	11
3.4.1	Warstwy backendu	11

3.4.2	Endpointy REST	11
3.5	Storage: PostgreSQL	11
3.6	Słownik zmiennych bazodanowych	12
3.7	Feature engineering	13
3.7.1	Przykładowe cechy dynamiczne	13
3.7.2	Cechy środowiskowe	14
3.7.3	Słownik zmiennych cechowych	14
3.8	Model i inferencja	15
3.8.1	Trening offline	15
3.8.2	Predykcja online	15
3.9	Policy engine / enforcement	16
3.10	DevOps i uruchomienie	16
3.11	Jakość i testy	16
4	Model finalny i raport wyników	16
4.1	Cel i zakres analizy modelu	16
4.2	Konfiguracja modelu finalnego	17
4.3	Jakie dane weszły do modelu	17
4.3.1	Poziom surowy	17
4.3.2	Poziom cech	17
4.4	Dobór zmiennych: proces i uzasadnienie	17
4.4.1	Przestrzeń kandydacka i finalny wybór	17
4.4.2	Kryteria doboru zmiennych	18
4.4.3	Co oznacza ten wybór w praktyce	18
4.5	Podział train/test i walidacja	18
4.5.1	Strategia podziału	18
4.5.2	Co oznaczają liczby w praktyce	18
4.5.3	Typ walidacji	18
4.6	Metryki ilościowe	19
4.6.1	Metryki z treningu (hold-out, GroupShuffleSplit)	19
4.6.2	Metryki ewaluacyjne (aktywny model)	19
4.7	Wizualizacje modelu	20
4.8	Interpretacja: co model rzeczywiście robi	21
4.8.1	Struktura decyzyjna drzewa: interpretacja warstw	21
4.8.2	Jak czytać diagram ważności cech	21
4.8.3	Znaczenie operacyjne (co to oznacza dla detekcji)	22
4.9	Reguły klasyfikacji (fragment)	22
4.10	Czy ewaluacja jest spójna z rzeczywistością?	22
4.10.1	Co jest spójne	22
4.10.2	Gdzie jest ryzyko przeszacowania	22

4.10.3	Najwazniejsze ograniczenie obecnego raportu	22
4.11	Threats to validity	23
4.12	Wniosek koncowy	23
5	Podsumowanie projektu	23
5.1	Co zostało osiągnięte	23
5.2	Wnioski praktyczne	24
5.3	Rekomendowane kolejne kroki	24

1 Wprowadzenie i motywacja

1.1 Kontekst problemu

Aplikacje webowe są stale narażone na automatyczny ruch generowany przez boty: masowe skanowanie zasobów, testowanie endpointów, tworzenie sztucznego ruchu lub działania fraudowe. Klasyczne podejścia ochronne oparte wyłącznie o reguły sieciowe i statyczne sygnatury przestają być wystarczające, ponieważ nowoczesne boty potrafią imitować część zachowań użytkownika.

1.2 Dlaczego behawioralna biometria myszy

Trajektoria kursora, rytm kliknięć, dynamika scrollowania i przejścia focus/blur niosą bogatą informację o sposobie interakcji człowieka z interfejsem. W odróżnieniu od pojedynczego eventu, analiza krótkiego okna zachowania:

- lepiej oddaje *styl* interakcji,
- utrudnia prostym botom przejście niezauważonym,
- daje możliwość szybkiej reakcji ochronnej jeszcze w trakcie sesji.

1.3 Cel projektu

Celem MVP było zbudowanie kompletnego pipeline'u *end-to-end*:

- frontend zbiera telemetrykę zachowania i fingerprint środowiska,
- backend zapisuje sesje i surowe eventy,
- moduł cech buduje reprezentację okien zachowania,
- model klasyfikuje `human` vs `bot`,
- silnik polityk zwraca akcję: `allow/observe/throttle/challenge/block`.

1.4 Zakres i świadome uproszczenia MVP

Wersja demonstracyjna została zaprojektowana profesjonalnie, ale celowo uproszczona:

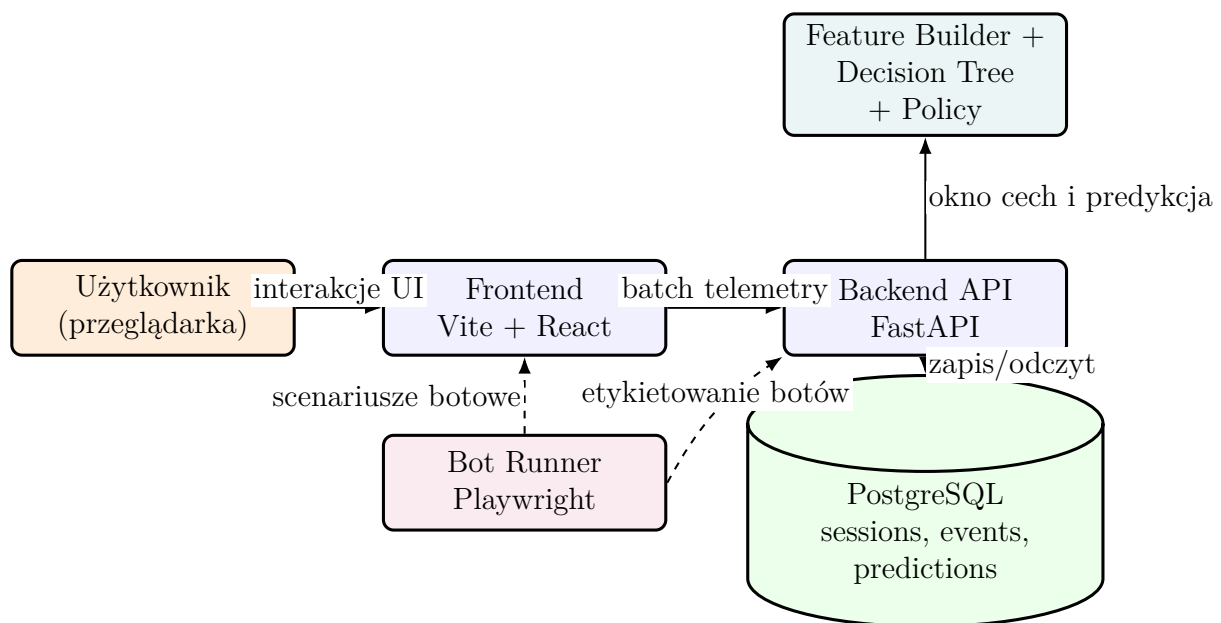
- inferencja modelu odbywa się w tym samym backendzie FastAPI (bez osobnego model service),
- predykcja działa na poziomie sesji i krótkich okien eventów,
- brak integracji z zewnętrznym WAF i blokadą IP (reakcja sesyjna),
- nacisk na czytelność kodu, odtwarzalność wyników i łatwą prezentację lokalną.

2 Architektura systemu

2.1 Widok komponentów

System składa się z czterech głównych części uruchamianych przez Docker Compose:

- **Frontend** (Vite + React + TypeScript) – zbiera telemetry użytkownika.
- **Backend** (FastAPI) – kolekcja eventów, predykcja, etykietowanie, API.
- **PostgreSQL** – trwale składowanie sesji, eventów, predykcji i akcji.
- **Bot Runner** (Playwright) – generowanie powtarzalnych scenariuszy botowych.

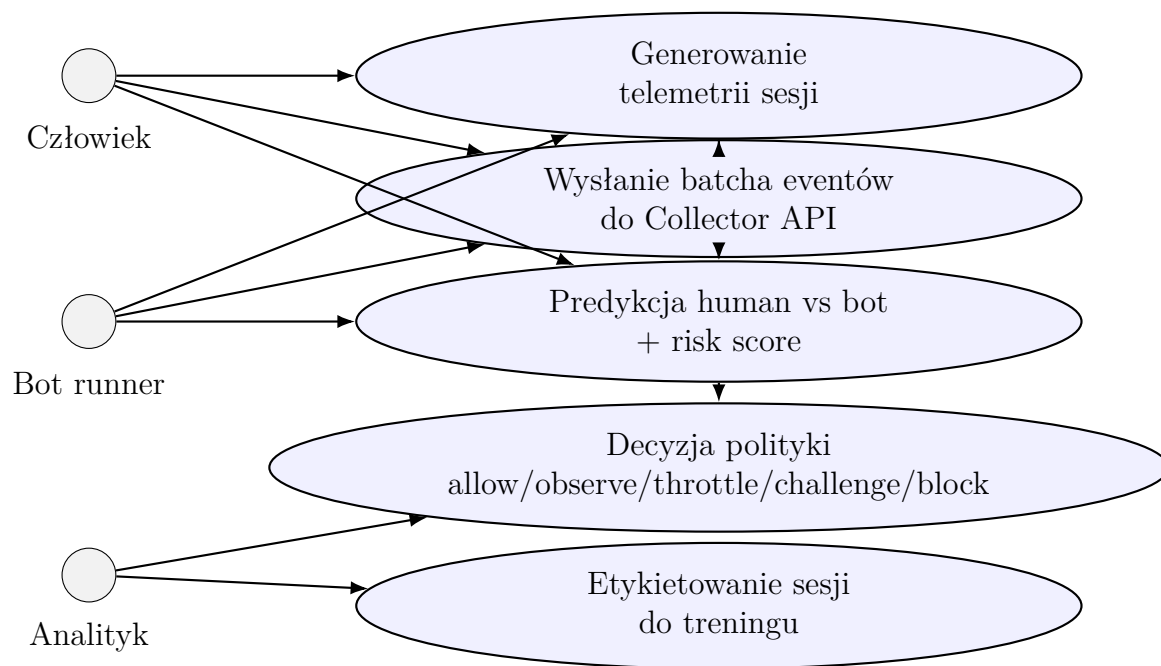


Rysunek 1: Architektura logiczna MVP

2.2 Przepływ danych end-to-end

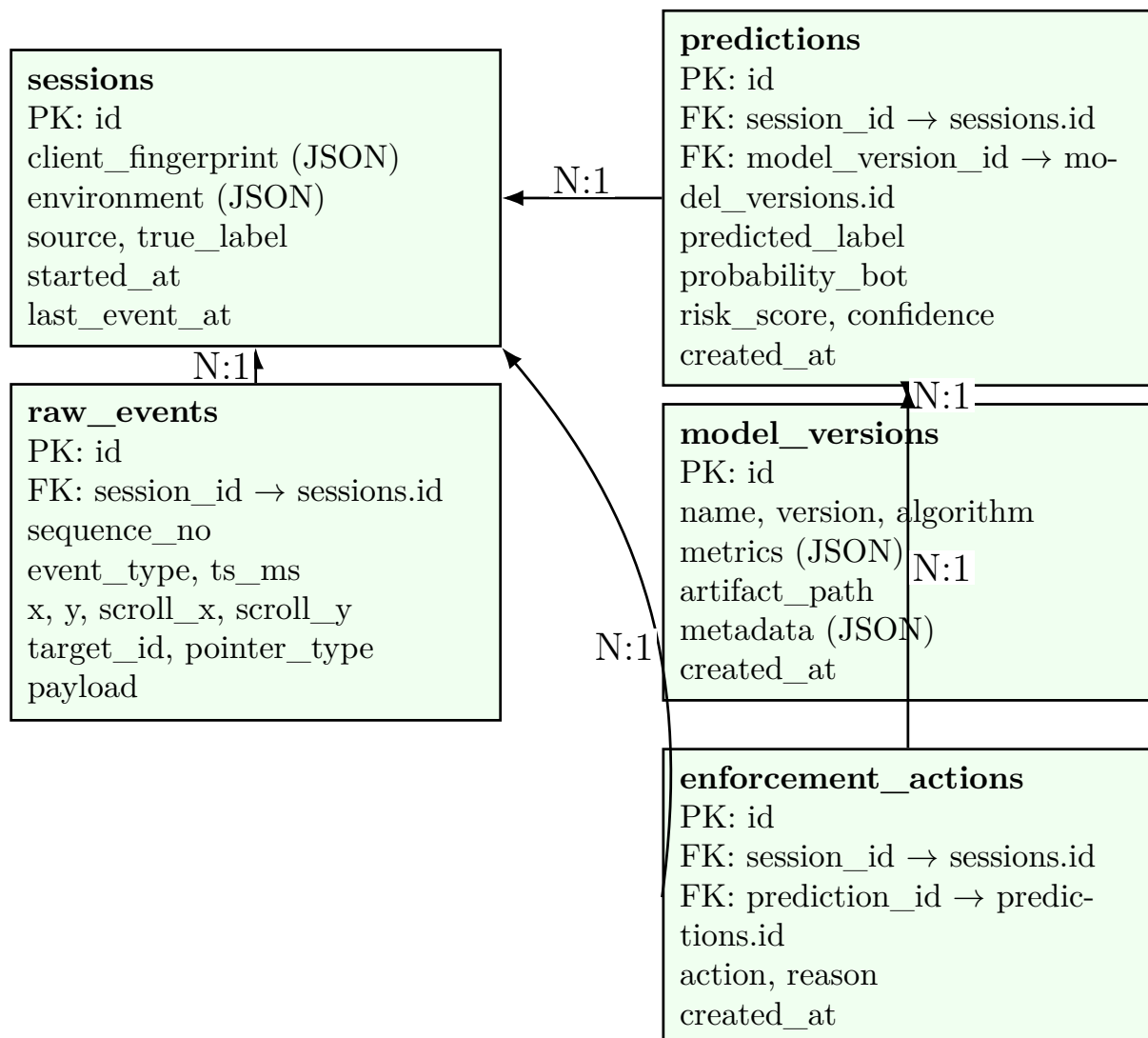
1. Frontend inicjuje sesję przez `POST /api/v1/sessions`.
2. Kolektor telemetry zbiera eventy i wysyła je batchami do `/api/v1/events/batch`.
3. Backend zapisuje surowe dane do tabel `sessions` i `raw_events`.
4. Dla żądania predykcji backend buduje cechy z okna eventów.
5. Model zwraca `predicted_label`, prawdopodobieństwo bota i `risk_score`.
6. Policy engine mapuje `risk_score` na akcję enforcement.
7. Wynik i akcja są zapisywane (`predictions`, `enforcement_actions`).

2.3 Diagram przypadków użycia



Rysunek 2: Główne przypadki użycia systemu

2.4 Model danych (ERD)



Rysunek 3: Relacyjny model danych PostgreSQL

3 Dokumentacja techniczna codebase

3.1 Struktura repozytorium

```
Projekt_ADAC/
  backend/
    app/
      api/routes/ # endpointy FastAPI
      core/ # konfiguracja, DB, logging
      models/ # ORM + schematy request/response
      repositories/ # dostep do danych
      services/ # feature engineering, policy, telemetry, trening danych
      ml/ # trening i ewaluacja modelu
      scripts/ # seed i eksport replay
      tests/ # testy jednostkowe
```

```
frontend/  
  src/  
    api/ # klient API  
    components/ # testbed UI  
    telemetry/ # collector + fingerprint + typy  
  bot_runner/  
    main.py # uruchamianie scenariuszy Playwright  
    scenarios.py # linear, human_like, adaptive, manual  
  docker-compose.yml  
  README.md  
  raport/ # dokumentacja LaTeX
```

3.2 Frontend testbed (Vite + React + TypeScript)

3.2.1 Rola frontendu

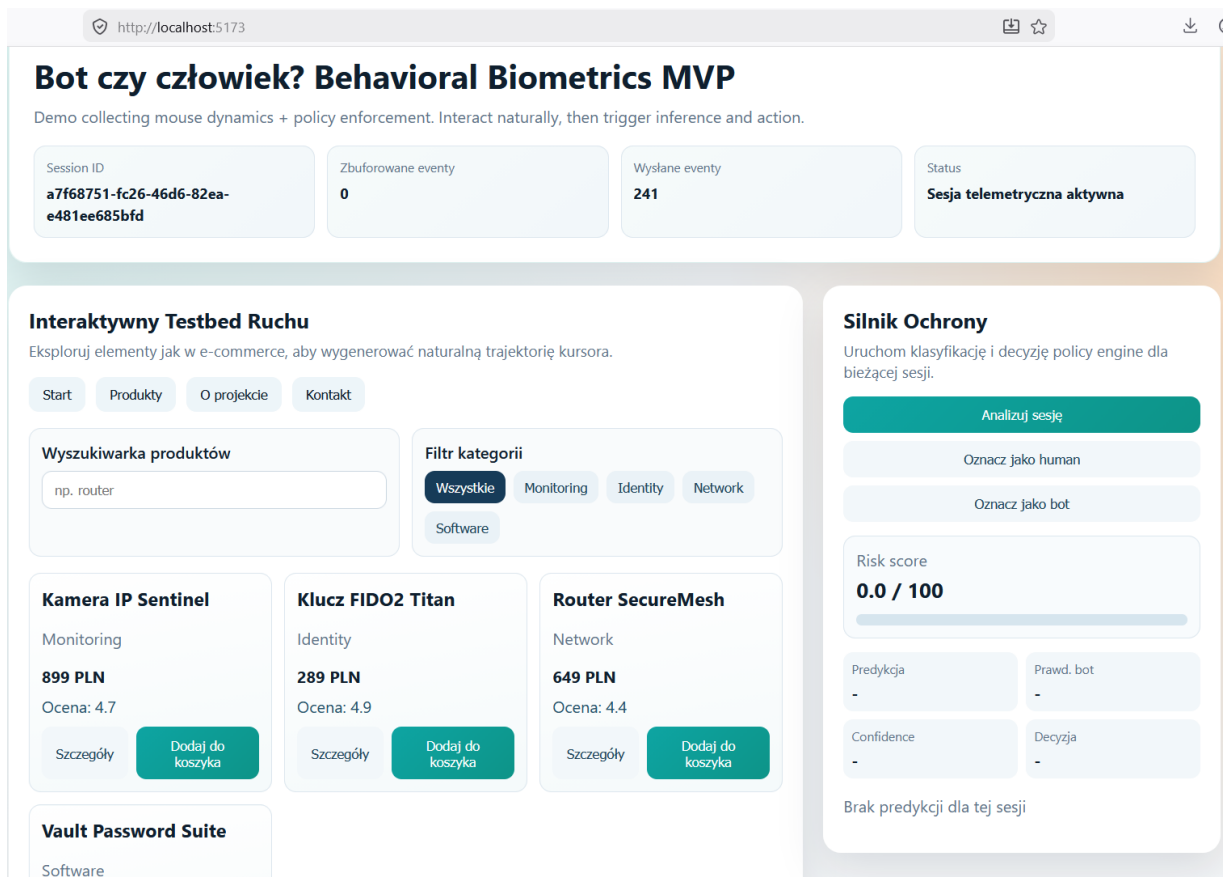
Frontend pełni podwójną funkcję:

- **generatora naturalnych interakcji użytkownika** (interfejs testowy),
- **sensora telemetrycznego** wysyłającego zdarzenia do backendu.

3.2.2 Wygląd i funkcje interfejsu

Widok frontendu pokazano na Rysunku 4. Układ ma trzy strefy:

- **Panel sesji telemetrycznej** (gora): `session id`, liczniki eventów (zbuforowane/wysłane) i status collector pipeline.
- **Interaktywny testbed ruchu** (lewa kolumna): menu, wyszukiwarka, filtry kategorii, karty produktowe i przyciski CTA. To celowo przypomina realistyczny scenariusz e-commerce.
- **Silnik ochrony** (prawa kolumna): uruchamianie predykcji, ręczne etykietowanie sesji (human/bot) i podgląd wyniku: `risk score`, `predykcja`, `confidence`, `decyzja`.



Rysunek 4: Frontend testbed na localhost: panel telemetryczny, strefa interakcji i panel policy engine

3.2.3 Zbierane eventy i batching

Kolektor (`frontend/src/telemetry/collector.ts`) rejestruje:

- `mousemove`,
- `click`,
- `scroll`,
- `focus`,
- `blur`,
- `viewport_enter/viewport_leave` (przez `visibilitychange`).

Wysyłka odbywa się partiami:

- `batchSize = 35`,
- okresowe `flush` co 2000 ms,
- `retry`: eventy wracają do bufora przy błędzie API.

3.2.4 Fingerprint klienta

Zbierane cechy srodowiskowe obejmują m.in.:

- `user_agent`, `language`, `timezone`,
- `screen_width/height`, `viewport_width/height`,
- `platform`, `pointer_type`,
- `webdriver`, `headless_hint`,
- `hardware_concurrency`, `device_memory`.

3.3 Pozyskanie danych treningowych (human vs bot)

3.3.1 Jak zbierano dane klasy human

Dane `human` były zbierane manualnie przez interakcje operatora z frontendem (Rysunek 4): ruch myszy po menu, filtrach i kartach, kliknięcia, scrollowanie i wpisywanie fraz w wyszukiwarce. Po zakończonej próbie sesje oznaczano przyciskiem `Oznacz jako human`.

3.3.2 Jak zbierano dane klas botowych

Sesje botowe generowano automatycznie przez `bot_runner`:

```
docker compose exec bot_runner python main.py --scenario linear --sessions 25 --record-video
docker compose exec bot_runner python main.py --scenario human_like --sessions 26 --record-video
docker compose exec bot_runner python main.py --scenario adaptive --sessions 50 --record-video
```

3.3.3 Udział scenariuszy botowych

Na poziomie sesji botowych przyjęto:

- `linear`: 25 sesji (24.75% klasy bot),
- `human_like`: 26 sesji (25.74% klasy bot),
- `adaptive`: 50 sesji (49.50% klasy bot).

Łącznie: 101 sesji botowych.

3.3.4 Charakterystyka botów i zamysł projektowy

- `linear`: ruch prosty i rytmiczny, niski realizm. Cel: łatwy baseline wykrywania prostych automatyzacji.
- `human_like`: krzywe Beziera, jitter i losowe opóźnienia. Cel: etap pośredni pomiędzy botem prostym i trudniejszym.

- **adaptive**: scenariusz kalibrowany profilem zachowan czlowieka (`/api/v1/sessions/human-behavior`) z dynamicznym tempem ruchu, pauzami, korektami toru i bardziej realistycznym typingiem. Cel: utrudnic klasyfikacje i przetestowac odpornosc modelu na boty zblizone dystrybucyjnie do klasy `human`.

3.4 Backend Collector API (FastAPI)

3.4.1 Warstwy backendu

- **API routes** – walidacja wejscia/wyjscia (Pydantic) i kontrakty REST.
- **Services** – logika domenowa (telemetria, cechy, polityki, model).
- **Repositories** – operacje bazodanowe (SQLAlchemy ORM).
- **ML module** – trening i ewaluacja modelu.

3.4.2 Endpointy REST

Endpoint	Metoda	Opis
<code>/api/v1/health</code>	GET	Healthcheck aplikacji.
<code>/api/v1/sessions</code>	POST	Utworzenie nowej sesji i zapis fingerprintu.
<code>/api/v1/events/batch</code>	POST	Przyjecie batcha eventow dla sesji.
<code>/api/v1/sessions/{session_id}/label</code>	POST	Etykietowanie sesji jako <code>human</code> lub <code>bot</code> .
<code>/api/v1/predict/{session_id}</code>	POST	Predykcja dla sesji + decyzja policy engine.
<code>/api/v1/predictions/{session_id}</code>	GET	Pobranie ostatniej predykcji sesji.
<code>/api/v1/sessions/{session_id}</code>	GET	Status sesji i metadane.
<code>/api/v1/sessions/human-behavior-profile</code>	GET	Profil statystyczny zachowan czlowieka do adaptacji bota.

3.5 Storage: PostgreSQL

Model danych sklada sie z pieciu tabel:

- `sessions` – glowna encja sesji,
- `raw_events` – surowy strumien interakcji,
- `predictions` – wyniki modelu,
- `model_versions` – rejestr wersji modelu i metryk,
- `enforcement_actions` – decyzje polityki ochronnej.

Relacje opisuje diagram ERD w Sekcji 2.

3.6 Słownik zmiennych bazodanowych

Tabela 2 pokazuje wszystkie zmienne przechowywane w bazie PostgreSQL. Kolumna czy wchodzi do modelu oznacza, czy dana zmienna jest wykorzystywana w pipeline cech (bezpośrednio lub przez transformacje).

Tabela 2: Słownik zmiennych bazodanowych: znaczenie i udział w modelu

nazwa_zmiennej	co zmienna oznacza (jak jest wyliczana)	czy wchodzi do modelu
sessions.id	identyfikator sesji (UUID).	NIE
sessions.source	zrodlo sesji (np. <code>frontend</code> / <code>bot_runner</code>).	NIE
sessions.true_label	etykieta referencyjna <code>human/bot</code> nadawana recznie lub automatycznie.	NIE
sessions.status	status procesu sesji (np. <code>collecting</code> , <code>predicted</code>).	NIE
sessions.client_fingerprint	surowy fingerprint klienta (JSON).	NIE
sessions.environment	surowe parametry srodowiskowe (JSON), zrodlo cech srodowiskowych.	TAK
sessions.created_at	czas utworzenia rekordu sesji.	NIE
sessions.updated_at	czas ostatniej aktualizacji sesji.	NIE
sessions.environment.user_agent	identyfikator przegladarki/OS; uzywany do flag pomocniczych.	TAK
sessions.environment.language	jezyk przegladarki.	NIE
sessions.environment.timezone	strefa czasowa klienta.	NIE
sessions.environment.screen_width	szerokosc ekranu klienta; skladnik <code>screen_area</code> .	TAK
sessions.environment.screen_height	wysokosc ekranu klienta; skladnik <code>screen_area</code> .	TAK
sessions.environment.viewport_width	szerokosc viewportu; skladnik <code>viewport_area</code> .	TAK
sessions.environment.viewport_height	wysokosc viewportu; skladnik <code>viewport_area</code> .	TAK
sessions.environment.webdriver	flaga automatyzacji przegladarki.	TAK
sessions.environment.headless_hint	sygnal trybu headless.	TAK
sessions.environment.hardware_concurrency	liczba logicznych rdzeni CPU raportowana przez przegladarke.	TAK
sessions.environment.device_memory	pamiec urzadzenia raportowana przez przegladarke.	TAK
raw_events.id	techniczny identyfikator eventu.	NIE
raw_events.session_id	klucz obcy do sesji.	NIE
raw_events.sequence_no	numer sekwencyjny eventu w batchu.	NIE
raw_events.event_type	typ eventu (<code>mousemove</code> , <code>click</code> , <code>scroll</code> , ...).	TAK
raw_events.ts_ms	timestamp eventu w milisekundach, baza do dynamiki.	TAK
raw_events.x	pozycja X kursora.	TAK
raw_events.y	pozycja Y kursora.	TAK
raw_events.scroll_x	przesuniecie poziome scrolla.	NIE
raw_events.scroll_y	przesuniecie pionowe scrolla, baza <code>scroll_velocity</code> .	TAK
raw_events.target_id	identyfikator celu eventu; baza <code>hover/target_revisit</code> .	TAK
raw_events.target_tag	tag HTML celu interakcji.	NIE
raw_events.target_class	klasy CSS celu interakcji.	NIE
raw_events.pointer_type	typ urzadzenia wskazujacego.	NIE
raw_events.in_viewport	flaga widocznosci eventu w viewport.	NIE

nazwa_zmiennej	co zmienna oznacza (jak jest wyliczana)	czy wchodzi do modelu
raw_events.payload	dodatkowy payload zdarzenia (JSON).	NIE
raw_events.created_at	czas zapisu eventu.	NIE
predictions.id	identyfikator predykcji.	NIE
predictions.session_id	sesja, dla której wykonano predykcje.	NIE
predictions.model_version_id	wersja modelu użyta do inferencji.	NIE
predictions.predicted_label	wynik klasyfikacji modelu (human/bot).	NIE
predictions.probability_bot	prawdopodobieństwo klasy bot .	NIE
predictions.confidence	pewność predykcji (maksimum prawdopodobieństwa).	NIE
predictions.risk_score	przeskalowany wynik ryzyka 0–100.	NIE
predictions.feature_snapshot	snapshot wektora cech wykorzystanych przy predykcji.	NIE
predictions.created_at	czas utworzenia predykcji.	NIE
model_versions.id	identyfikator wersji modelu.	NIE
model_versions.name	nazwa modelu.	NIE
model_versions.version	oznaczenie wersji (np. timestamp).	NIE
model_versions.algorithm	algorytm (Decision Tree).	NIE
model_versions.feature_set_version	wersja zestawu cech.	NIE
model_versions.metrics	metryki treningowe/ewaluacyjne (JSON).	NIE
model_versions.artifact_path	ścieżka do artefaktu modelu.	NIE
model_versions.model_metadata	metadane modelu (JSON).	NIE
model_versions.is_active	flaga aktywnej wersji modelu.	NIE
model_versions.created_at	czas rejestracji wersji modelu.	NIE
enforcement_actions.id	identyfikator decyzji polityki.	NIE
enforcement_actions.session_id	sesja, której dotyczy decyzja.	NIE
enforcement_actions.prediction_id	predykcja powiązana z decyzją.	NIE
enforcement_actions.action	decyzja (allow/observe/throttle/challenge/block).	NIE
enforcement_actions.reason	uzasadnienie decyzji polityki.	NIE
enforcement_actions.blocked_until	koniec blokady czasowej (jeśli dotyczy).	NIE
enforcement_actions.action_metadata	dodatkowe metadane decyzji (JSON).	NIE
enforcement_actions.created_at	czas zapisu decyzji.	NIE

3.7 Feature engineering

Pipeline cech (`backend/app/services/feature_engineering.py`) liczy cechy z okna eventów posortowanych po czasie (`ts_ms`), a nie z pojedynczego eventu.

3.7.1 Przykładowe cechy dynamiczne

- trajektoria i geometria: `path_length`, `displacement`, `path_efficiency`,
- dynamika ruchu: `speed_mean/std/max`, `acc_mean/std/max`, `jerk_mean/std/max`,
- mikrozachowania: `micro_pause_count`, `correction_rhythm`, `overshoot_count`,

- kliknięcia: `click_interval_mean/std`, `hover_before_click_mean`, `target_revisit_count`,
- kontekst sekwencji: `event_type_entropy`, `unique_event_types`,
- scroll i focus: `scroll_velocity_mean/std`, `focus_blur_transitions`.

3.7.2 Cechy srodowiskowe

Pipeline zawiera także cechy środowiskowe, jednak w finalnym modelu drzewa są one **domyslnie wyłączone z treningu** (`feature_set=behavioral`), aby ograniczyć ryzyko data leakage (np. trywialny podział po `viewport_area`).

3.7.3 Słownik zmiennych cechowych

Tabela 3 zawiera komplet zmiennych cechowych budowanych w pipeline i informacje, czy dana zmienna wchodzi do finalnego modelu.

Tabela 3: Słownik zmiennych cechowych: nazwa, znaczenie i użycie w modelu finalnym

nazwa_zmiennej	co zmienna oznacza (jak jest wyliczana)	czy wchodzi do modelu
<code>event_count</code>	liczba wszystkich eventów w oknie czasowym.	TAK
<code>window_duration_ms</code>	czas trwania okna: <code>ts_last - ts_first</code> .	TAK
<code>mousemove_count</code>	liczba eventów <code>mousemove</code> .	TAK
<code>click_count</code>	liczba eventów <code>click</code> .	TAK
<code>scroll_count</code>	liczba eventów <code>scroll</code> .	TAK
<code>focus_blur_transitions</code>	suma liczby eventów <code>focus</code> i <code>blur</code> .	TAK
<code>viewport_enter_count</code>	liczba eventów <code>viewport_enter</code> .	TAK
<code>viewport_leave_count</code>	liczba eventów <code>viewport_leave</code> .	TAK
<code>path_length</code>	suma odległości euklidesowych pomiędzy kolejnymi punktami kursora.	TAK
<code>displacement</code>	odległość między pierwszym i ostatnim punktem kursora w oknie.	TAK
<code>path_efficiency</code>	współczynnik <code>displacement / max(path_length, 1)</code> .	TAK
<code>speed_mean</code>	średnia prędkość kursora pomiędzy kolejnymi punktami (<code>distance/dt</code>).	TAK
<code>speed_std</code>	odchylenie standardowe prędkości kursora.	TAK
<code>speed_max</code>	maksymalna prędkość kursora w oknie.	TAK
<code>acc_mean</code>	średnie przyspieszenie: <code>delta(speed)/dt</code> .	TAK
<code>acc_std</code>	odchylenie standardowe przyspieszenia.	TAK
<code>acc_max</code>	maksymalne przyspieszenie w oknie.	TAK
<code>jerk_mean</code>	średni jerk: <code>delta(acc)/dt</code> .	TAK
<code>jerk_std</code>	odchylenie standardowe jerk.	TAK
<code>jerk_max</code>	maksymalny jerk w oknie.	TAK
<code>curvature_mean</code>	średnia krzywizna toru: <code> delta(kat) / max(distance, 1)</code> .	TAK
<code>curvature_std</code>	odchylenie standardowe krzywizny toru.	TAK

nazwa_zmiennej	co zmienna oznacza (jak jest wyliczana)	czy wchodzi do modelu
angle_change_mean	srednia zmiana kata ruchu pomiedzy kolejnymi segmentami.	TAK
angle_change_std	odchylenie standardowe zmian kata ruchu.	TAK
micro_pause_count	liczba segmentow z bardzo niska predkoscia (<code>speed < 40 px/s</code>).	TAK
correction_rhythm	liczba mocnych korekt kierunku (prog zmiany kata > 1.1 rad).	TAK
click_interval_mean	sredni odstep czasowy miedzy kolejnymi kliknieciami.	TAK
click_interval_std	odchylenie standardowe odstepow miedzy kliknieciami.	TAK
hover_before_click_mean	sredni czas hover przed kliknieciem, liczony lokalnie przy celu klikniecia.	TAK
dwell_time_mean	sredni czas przebywania na segmentach ($1000/\max(\text{speed}, 1)$).	TAK
overshoot_count	liczba przypadkow overshoot (rewersy kierunku przed kliknieciem).	TAK
target_revisit_count	liczba celow klikanych wielokrotnie w tej samej sesji okna.	TAK
scroll_velocity_mean	srednia predkosc scrolla: $ \Delta(\text{scroll}_y) /\Delta t$.	TAK
scroll_velocity_std	odchylenie standardowe predkosci scrolla.	TAK
unique_event_types	liczba unikalnych typow eventow w oknie.	TAK
event_type_entropy	entropia Shannona rozkladu typow eventow w oknie.	TAK
webdriver_flag	flaga 0/1: czy <code>navigator.webdriver</code> jest prawda.	NIE
headless_flag	flaga 0/1: wykrycie sygnalu headless na podstawie <code>user-agent</code> .	NIE
viewport_area	pole viewportu: <code>viewport_width * viewport_height</code> .	NIE
screen_area	pole ekranu: <code>screen_width * screen_height</code> .	NIE
viewport_screen_ratio	iloraz <code>viewport_area / max(screen_area, 1)</code> .	NIE
hardware_concurrency	liczba logicznych rdzeni CPU raportowana przez przeglądarkę.	NIE
device_memory	pamięć urządzenia raportowana przez przeglądarkę.	NIE

3.8 Model i inferencja

3.8.1 Trening offline

Skrypt `python -m app.ml.train` realizuje:

- budowe `DataFrame` z okien (`window_size`, `stride`),
- podzial `train/test` z grupowaniem po `session_id` (`GroupShuffleSplit`),
- trening `DecisionTreeClassifier` z `class_weight=balanced`,
- zapis artefaktow: model, metryki, ROC, confusion matrix, drzewo i reguly.

3.8.2 Predykcja online

Inferencja dziala przez `ModelManager`:

- ładuje aktywny artefakt modelu (`active_model.joblib`),
- zwraca `predicted_label`, `probability_bot`, `confidence`, `risk_score`,
- przy braku modelu używa fallbacku heurystycznego.

3.9 Policy engine / enforcement

Mapowanie ryzyka (`backend/app/services/policy_engine.py`):

- `risk < 25`: allow,
- `25-50`: observe,
- `50-75`: throttle,
- `75-90`: challenge,
- `>= 90`: block.

3.10 DevOps i uruchomienie

Środowisko działa lokalnie przez Docker Compose:

- `frontend` – port 5173,
- `backend` – port 8000,
- `db` – port 5432,
- `bot_runner` – kontener narzędziowy do generacji sesji.

3.11 Jakosc i testy

- testy jednostkowe backendu (`backend/app/tests`),
- walidacja kontraktów API przez schematy Pydantic,
- separacja odpowiedzialności: API/Services/Repositories/ML.

4 Model finalny i raport wynikow

4.1 Cel i zakres analizy modelu

W tej sekcji dokumentujemy finalny model klasyfikacji `human` vs `bot`, z naciskiem na:

- jakie dane i cechy weszły do treningu,
- jak wykonano podział na trening i test,
- jak przeprowadzono walidacje i ewaluacje,
- na ile wyniki odpowiadają rzeczywistemu działaniu systemu.

4.2 Konfiguracja modelu finalnego

Finalna wersja modelu to drzewo decyzyjne trenowane na cechach behawioralnych:

- algorytm: `DecisionTreeClassifier`,
- parametry okna: `window_size = 120, stride = 80`,
- hiperparametry: `max_depth = 12, min_samples_leaf = 2`,
- `class_weight = balanced`,
- `feature_set = behavioral` (bez cech srodowiskowych).

4.3 Jakie dane weszly do modelu

4.3.1 Poziom surowy

Wejsciem sa sesje zapisane w PostgreSQL (`sessions, raw_events`), z etykieta referencyjna `true_label` nadana przez operatora. Kazde okno zawiera uporządkowany po czasie fragment zachowania, a nie pojedynczy event.

4.3.2 Poziom cech

Z okna eventow budowane sa cechy opisujace:

- dynamike ruchu kursora: `speed_*`, `acc_*`, `jerk_*`,
- geometrie toru: `path_length`, `path_efficiency`, `curvature_*`,
- mikrozachowania: `micro_pause_count`, `correction_rhythm`, `overshoot_count`,
- wzorzec klikniec i scrolla: `click_interval_*`, `hover_before_click_mean`, `scroll_velocity_*`,
- entropie sekwencji: `event_type_entropy`, `unique_event_types`.

W finalnym modelu swiadomie pominieto cechy srodowiskowe (np. `viewport_area`), zeby ograniczyc ryzyko sztucznego zawyzania wyniku przez data leakage.

4.4 Dobor zmiennych: proces i uzasadnienie

4.4.1 Przestrzen kandydacka i finalny wybor

Proces startowal od pelnego slownika cech wyliczanego przez pipeline (`FEATURE_COLUMNS`) opisany w Tabeli 3. W praktyce byly to:

- 43 cechy kandydackie laczenie,
- 7 cech srodowiskowych,
- 36 cech behawioralnych.

Do finalnego modelu weszly cechy behawioralne (`feature_set=behavioral`), natomiast cechy srodowiskowe zostaly wylaczone z trenowania drzewa.

4.4.2 Kryteria doboru zmiennych

Selekcja była oparta o trzy kryteria:

- **odpornosc na leakage:** odciecie sygnałów typu `viewport_area` i `screen_area`, które mogą rozdzielać klasy na skroty, bez realnej analizy zachowania,
- **związek z mikrodynamiką ruchu:** preferencja dla cech opisujących kinetykę i geometrie toru (`acc_*`, `speed_*`, `curvature_*`),
- **interpretowalność:** drzewo z tak dobranym zbiorem cech daje reguły decyzyjne, które można odczytać i wyjaśnić.

4.4.3 Co oznacza ten wybór w praktyce

Model finalny klasyfikuje sesje głównie na podstawie *jak* porusza się kursor i jak wygląda rytm interakcji, a nie *na czym* sesja jest wykonywana. To zwiększa wiarygodność demonstracji, bo decyzja nie wynika z jednego artefaktu środowiskowego, tylko z profilu zachowania.

4.5 Podział train/test i walidacja

4.5.1 Strategia podziału

Podział został wykonany przez `GroupShuffleSplit` z `test_size=0.25` i grupowaniem po `session_id`. To kluczowe, bo:

- okna z tej samej sesji nie trafiają jednocześnie do train i test,
- test sprawdza uogólnienie na nieznane sesje, a nie tylko nieznane okna.

4.5.2 Co oznaczają liczby w praktyce

Z raportu metryk treningowych (Tabela 4 i Tabela 5) wynika, że zbiór testowy miał 136 okien ($TN+FP+FN+TP = 74+1+6+55$). W ewaluacji aktywnego modelu analizowano 644 okna i 202 sesje (Tabela 7).

4.5.3 Typ walidacji

W MVP użyto pojedynczego hold-out splitu grupowego (jedna próbka walidacyjna). To dobre rozwiązanie demonstracyjne, ale nie zastępuje pełnej walidacji krzyżowej np. `GroupKFold`.

4.6 Metryki ilosciowe

4.6.1 Metryki z treningu (hold-out, GroupShuffleSplit)

Metryka	Wartosc
Precision	0.9821
Recall	0.9016
F1-score	0.9402
ROC-AUC	0.9442
False Positive Rate (human)	0.0133

Tabela 4: Metryki modelu na zbiorze testowym podczas treningu

Element	Wartosc
TN	74
FP	1
FN	6
TP	55

Tabela 5: Confusion matrix (trening)

4.6.2 Metryki ewaluacyjne (aktywny model)

Metryka	Wartosc
Precision	0.9940
Recall	0.9651
F1-score	0.9794
ROC-AUC	0.9815
False Positive Rate (human)	0.0021

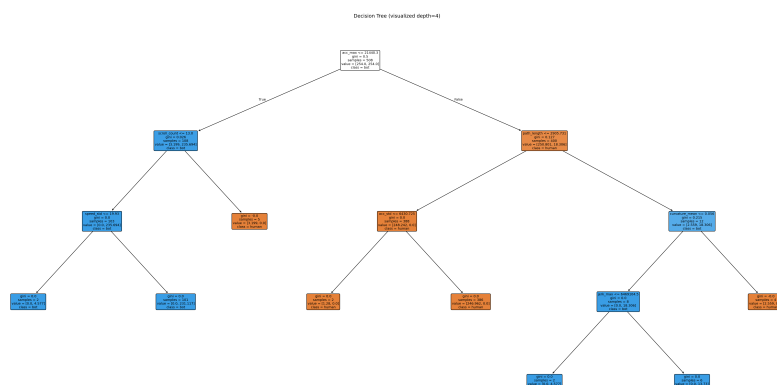
Tabela 6: Metryki z raportu ewaluacyjnego

Wskaznik	Wartosc
Liczba probek	644
Liczba sesji	202
Avg events to detect bot	122.2574
Median events to detect bot	120.0000
Detection delay count	101
Unseen bot recall count	0

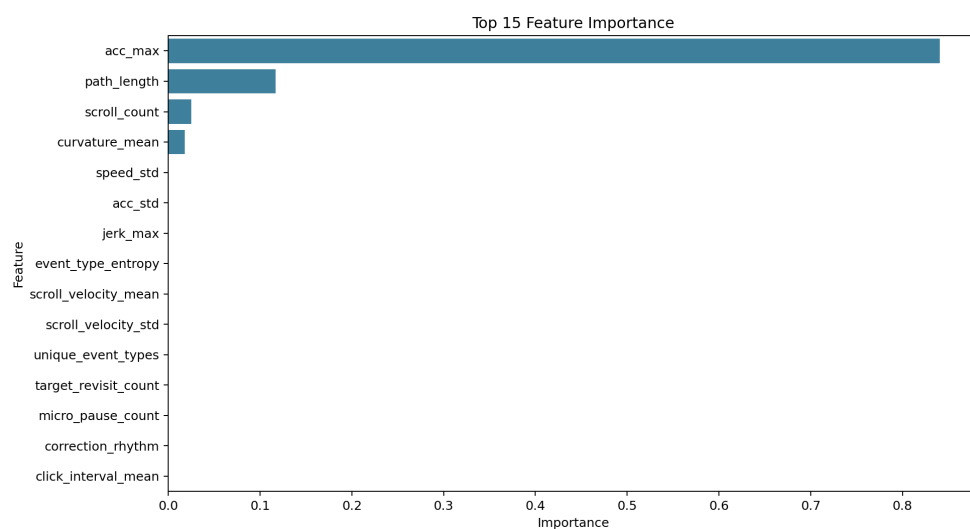
Tabela 7: Statystyki operacyjne ewaluacji

Wyniki z testu hold-out (Tabela 4) sa mocne (F1=0.9402, ROC-AUC=0.9442) przy niskim false positive rate dla klasy human (0.0133).

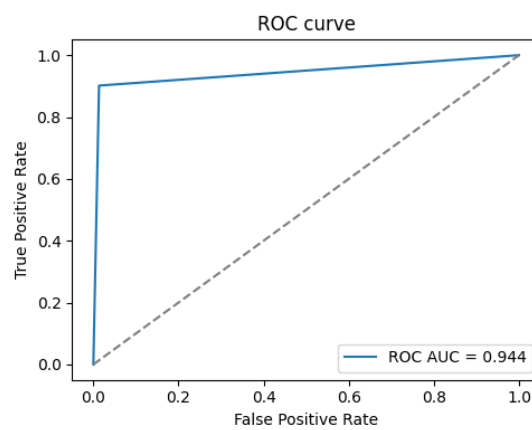
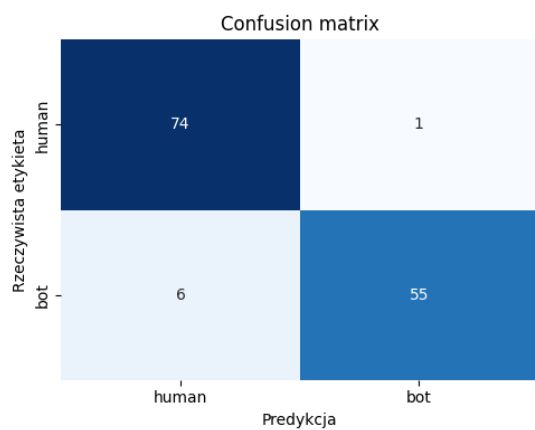
4.7 Wizualizacje modelu



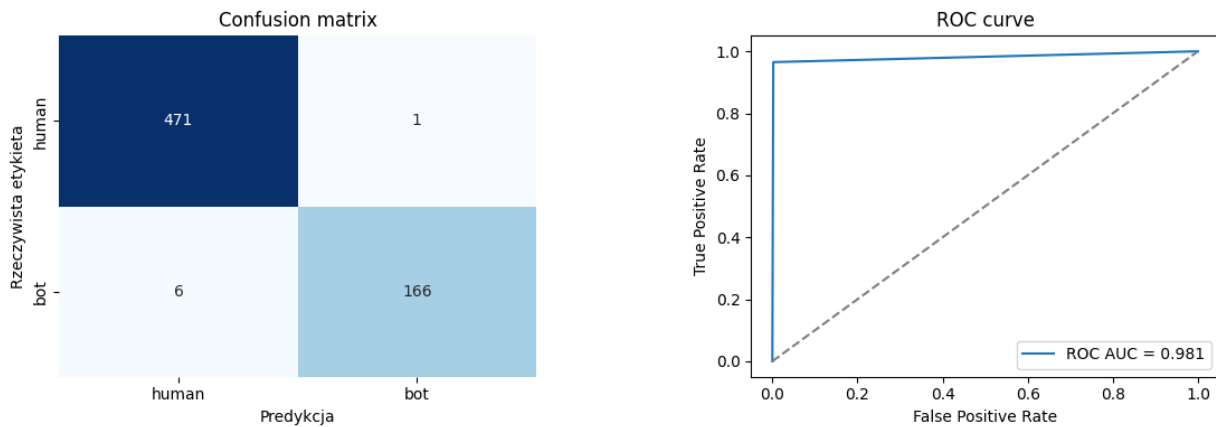
Rysunek 5: Diagram drzewa decyzyjnego (warstwy najwyższego poziomu)



Rysunek 6: Waznosc cech (feature importance) dla modelu finalnego



Rysunek 7: Confusion matrix oraz ROC dla zbioru testowego w treningu



Rysunek 8: Confusion matrix oraz ROC w ewaluacji aktywnego modelu

4.8 Interpretacja: co model rzeczywiscie robi

Rysunek 5 pokazuje, ze model jest interpretowalny i mozna odczytac jawne warunki decyzji. Rysunek 6 potwierdza, ze dominujaca cecha to nadal `acc_max`, a kolejne sygnały (np. `path_length`, `event_type_entropy`) sa uzupełniające.

4.8.1 Struktura decyzyjna drzewa: interpretacja warstw

Na podstawie Rysunku 5 i pełnych reguł można odczytać następującą logikę:

- **Wezeł główny (`acc_max`):** pierwszy podział oddziela sesje o niskich i wysokich pikach przyspieszenia kursora.
- **Galaz niskiego `acc_max`:** kolejne warunki `scroll_count` i `speed_std` prowadzi głównie do klasy `bot`. Intuicyjnie: ruch jest bardziej wygładzony i mniej impulsywny.
- **Galaz wysokiego `acc_max`:** podziały po `path_length` i `acc_std` częściej prowadzi do klasy `human`, bo odzwierciedlają bardziej zrywny i korekcyjny tor ruchu.
- **Wezły głębsze (`curvature_mean`, `jerk_max`)** doprecyzowują decyzje dla przypadków granicznych, gdzie sama dynamika pierwszego rzędu nie wystarcza.

4.8.2 Jak czytać diagram ważności cech

Rysunek 6 przedstawia ważności typu impurity-based (spadek nieczystości węzłów, Gini). Interpretacja:

- wysoka ważność `acc_max` oznacza, że ta cecha najczęściej daje największy przyrost informacji przy podziałach,
- niezerowa rola `path_length` i `event_type_entropy` pokazuje, że model korzysta także z geometrii toru i różnorodności sekwencji eventów,
- niska ważność wielu cech nie oznacza, że są globalnie niepotrzebne; oznacza, że w aktualnej próbie były rzadziej wybierane jako najlepszy podział.

4.8.3 Znaczenie operacyjne (co to oznacza dla detekcji)

Struktura drzewa jest spojna z celem systemu:

- sesje o zbyt gładkiej mikrodynamice są przesuwane w stronę **bot**,
- sesje z naturalnymi korektami i zrywami ruchu są przesuwane w stronę **human**,
- policy engine otrzymuje wynik oparty o zachowanie, a nie o pojedynczy artefakt środowiskowy.

4.9 Reguly klasyfikacji (fragment)

Model pozostaje interpretowalny – można odczytać reguły decyzyjne:

Listing 1: Przykładowy fragment reguł drzewa decyzyjnego

```
|--- acc_max <= 21440.2998
| |--- scroll_count <= 13.0000
| | |--- speed_std <= 19.9304 -> class: bot
|--- acc_max > 21440.2998
| |--- path_length <= 2905.7310
| | |--- acc_std > 6430.7249 -> class: human
```

Listing 1 jest bezpośrednim dopełnieniem Rysunku 5: po podziale na `acc_max` drzewo doprecyzowuje decyzje przez `scroll_count`, `speed_std`, `path_length` i `acc_std`. Taka sekwencja warunków jest czytelna biznesowo i łatwa do audytu.

4.10 Czy ewaluacja jest spojna z rzeczywistoscia?

4.10.1 Co jest spójne

- Hold-out test (Rysunek 7) i metryki z Tabeli 4 pokazują, że model dobrze odróżnia klasy na nieznanymi sesjach testowych.
- Metryka `detection delay` (Tabela 7) daje praktyczny sygnał, po ilu eventach system zwykle wykrywa bota.

4.10.2 Gdzie jest ryzyko przeszacowania

- Ewaluacja aktywnego modelu (Tabela 6, Rysunek 8) liczona jest na całym zbiorze oznaczonych okien, więc zawiera również dane wykorzystane do treningu.
- To oznacza, że wyniki ewaluacyjne są bardziej optymistyczne niż hold-out i nie powinny być jedynym kryterium oceny generalizacji.

4.10.3 Najważniejsze ograniczenie obecnego raportu

W Tabeli 7 `unseen_bot_recall` ma `count=0`, czyli w aktualnym zestawie nie ma osobnej, niewidzianej rodziny botów do twardego testu OOD. Dlatego aktualny raport dobrze opisuje zachowanie modelu *w ramach obecnej dystrybucji*, ale nie zamyka tematu odporności na nowe klasy botów.

4.11 Threats to validity

Ponizej wskazano glowne ryzyka interpretacyjne wynikow:

- **Internal validity:** mimo wykluczenia czesci cech srodowiskowych z podzialow drzewa, pozostaje ryzyko, ze model wykorzystuje sygnaly specyficzne dla aktualnego UI i sposobu testowania.
- **Construct validity:** etykieta `human` opiera sie na recznym oznaczaniu sesji, co jest praktyczne, ale moze zawierac szum (np. sesja czlowieka z nienaturalnym rytmem).
- **External validity:** dane pochodza glownie z lokalnego srodowiska (`localhost`), ograniczonej liczby urzadzen/przegladowek i kontrolowanych scenariuszy.
- **Conclusion validity:** zastosowano pojedynczy hold-out split grupowy, bez wielokrotnej walidacji (`GroupKFold`) i bez przedzialow ufności metryk.
- **OOD validity:** brak niezaleznej rodziny botow w metryce `unseen_bot_recall (count=0)` utrudnia twarda ocene odpornosci na nowe techniki automatyzacji.

Wniosek praktyczny: metryki sa mocne dla aktualnej dystrybucji danych, ale produkcyjna pewnosc wymaga dalszego rozszerzania zbioru i walidacji wielokrotnej.

4.12 Wniosek koncowy

Model finalny jest mocny i interpretowalny na aktualnym zbiorze (patrz Tabela 4 i Rysunek 5), jednak dla pelnej zgodnosci z warunkami produkcyjnymi potrzebuje kolejnego kroku: oddzielnego testu na niewidzianych scenariuszach botowych oraz walidacji wielokrotnej na poziomie sesji.

5 Podsumowanie projektu

5.1 Co zostalo osiagniete

W ramach MVP dostarczono działający system *end-to-end*, który:

- zbiera zachowania użytkownika w aplikacji webowej,
- buduje cechy behawioralne z krótkich okien czasowych,
- klasyfikuje sesje `human` vs `bot`,
- generuje decyzję enforcement na podstawie `risk_score`,
- umożliwia trenowanie i ewaluację modelu na danych lokalnych.

5.2 Wnioski praktyczne

- Podejście behawioralne daje dobrą wykrywalność nawet przy botach imitujących człowieka.
- Interpretowalne drzewo decyzyjne ułatwia prezentację i audyt decyzji modelu.
- Największy wpływ nadal mają cechy dynamiki ruchu (zwłaszcza `acc_max`), więc doskonalenie botów powinno iść w stronę realistycznego modelowania nagłych zmian tempa.

5.3 Rekomendowane kolejne kroki

1. Rozszerzyć zbiór o nowe rodziny botów i sesje *unseen*.
2. Dodać walidację czasową (time-based split) oraz monitoring driftu cech.
3. Uzupełnić policy engine o reputację źródeł i denylistę.
4. Rozważyć model sekwencyjny (np. Transformer/RNN) jako warstwę drugą.
5. Dodać dashboard operacyjny do przeglądu sesji i interpretacji predykcji.